

Differential Dynamic Programming (DDP)

Optimization and Learning for Robot Control

RCI Lab

Kyung Hee University

February 17, 2026

1. Problem Formulation: Nonlinear Systems

Objective: Minimize cost for a nonlinear system.

$$\min_U \sum_{t=0}^{N-1} l(x_t, u_t) + l_f(x_N)$$

subject to $x_{t+1} = f(x_t, u_t)$, $x_0 = x_{\text{init}}$

Key Idea: Since the dynamics f are nonlinear, we cannot solve this in one shot like LQR. Instead, we use an **Iterative Approach** (Newton's Method for Trajectories):

- 1 Start with an initial guess for control sequence U .
- 2 Linearize dynamics and quadraticize cost around the current trajectory.
- 3 Solve LQR to find the optimal variation δu .
- 4 Update trajectory and repeat until convergence.

2. The Q-Function Expansion

Let $V(x, t)$ be the optimal cost-to-go. We define the Q-function at step t :

$$Q(x, u) = l(x, u) + V(f(x, u), t + 1)$$

To improve our trajectory, we compute the ****Second-order Taylor Expansion**** of Q around the current nominal point $(\bar{x}, \bar{u})$:

$$Q(\bar{x} + \delta x, \bar{u} + \delta u) \approx Q_{nom} + \begin{bmatrix} Q_x^T & Q_u^T \end{bmatrix} \begin{bmatrix} \delta x \\ \delta u \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \delta x^T & \delta u^T \end{bmatrix} \begin{bmatrix} Q_{xx} & Q_{xu} \\ Q_{ux} & Q_{uu} \end{bmatrix} \begin{bmatrix} \delta x \\ \delta u \end{bmatrix}$$

Derivatives (Chain Rule):

- $Q_x = l_x + f_x^T V'_x$
- $Q_u = l_u + f_u^T V'_x$
- $Q_{xx} = l_{xx} + f_x^T V'_{xx} f_x + V'_x \cdot f_{xx}$ (The tensor term f_{xx} makes it DDP)
- $Q_{uu} = l_{uu} + f_u^T V'_{xx} f_u + V'_x \cdot f_{uu}$

3. Optimal Control Update

We want to find δu^* that minimizes the quadratic model of Q .

$$\delta u^* = \arg \min_{\delta u} Q(\bar{x} + \delta x, \bar{u} + \delta u)$$

Taking the gradient w.r.t δu and setting to zero:

$$Q_u + Q_{uu}\delta u + Q_{ux}\delta x = 0$$

Control Law:

$$\delta u^* = \underbrace{-Q_{uu}^{-1}Q_u}_k + \underbrace{-Q_{uu}^{-1}Q_{ux}}_K \delta x$$

k (Feedforward) K (Feedback)

- k : Open-loop modification to improve the trajectory.
- K : Feedback gain to stabilize the trajectory.

4. Value Function Propagation

Now substitute δu^* back into the Q-function expansion to update $V(x, t)$.

$$V(\bar{x} + \delta x) \approx V_{nom} + \Delta V + V_x^T \delta x + \frac{1}{2} \delta x^T V_{xx} \delta x$$

Recursive Updates (Backward Pass):

- $V_x = Q_x + K^T Q_{uu} k + K^T Q_u + Q_{xu}^T k$ (Simplified: $Q_x - Q_{xu} Q_{uu}^{-1} Q_u$)
- $V_{xx} = Q_{xx} + K^T Q_{uu} K + 2K^T Q_{ux}$ (Simplified: $Q_{xx} - Q_{xu} Q_{uu}^{-1} Q_{ux}$)
- $\Delta V = -\frac{1}{2} k^T Q_{uu} k$ (Expected cost reduction)

We propagate V_x and V_{xx} from $t = N$ down to 0.

5. Regularization (Levenberg-Marquardt)

Problem: Q_{uu} might not be positive definite (e.g., if the system is unstable or cost is non-convex). Then Q_{uu}^{-1} fails or leads to a maximum.

Solution: Add a damping term μI .

$$\bar{Q}_{uu} = Q_{uu} + \mu I$$

Adaptive Scheme:

- If backward pass fails ($\bar{Q}_{uu} \not\succeq 0$), increase μ ($\mu \leftarrow \mu \times \Delta$).
- If forward pass succeeds (Cost decreases), decrease μ ($\mu \leftarrow \mu / \Delta$).
- This makes DDP behave like Gradient Descent (robust) when far from solution, and Newton's Method (fast) when close.

6. Forward Pass with Line Search

We update the trajectory using the computed gains.

$$u_{new} = \bar{u} + \alpha k + K(x_{new} - \bar{x})$$

Line Search (α): Start with $\alpha = 1$ (Full Newton Step). If the actual cost J_{new} does not decrease as expected, reduce α (e.g., 0.5, 0.25...).

Why? The quadratic model is only an approximation. Large steps might leave the "trust region" where the approximation is valid.

7. Derivatives for Robot Dynamics

To implement DDP for a robot $M(q)\ddot{q} + h(q, \dot{q}) = \tau$, we need derivatives f_x, f_u .

State Space: $x = [q, \dot{q}]^T$, $u = \tau$.

$$\dot{x} = \begin{bmatrix} \dot{q} \\ M^{-1}(\tau - h) \end{bmatrix}$$

Analytical Derivatives:

$$\frac{\partial f}{\partial x} = \begin{bmatrix} 0 & I \\ -M^{-1}\frac{\partial h}{\partial q} & -M^{-1}\frac{\partial h}{\partial \dot{q}} \end{bmatrix}, \quad \frac{\partial f}{\partial u} = \begin{bmatrix} 0 \\ M^{-1} \end{bmatrix}$$

Note: We usually ignore $\frac{\partial M}{\partial q}$ terms in iLQR for simplicity.

8. Handling Discrete Time

Robots are controlled in discrete time steps Δt . If using **Forward Euler**: $x_{t+1} = x_t + f(x_t, u_t)\Delta t$.

Discrete Derivatives:

$$A_t = \frac{\partial x_{t+1}}{\partial x_t} = I + \frac{\partial f}{\partial x} \Delta t$$

$$B_t = \frac{\partial x_{t+1}}{\partial u_t} = \frac{\partial f}{\partial u} \Delta t$$

These matrices A_t, B_t are what we plug into the DDP backward pass.

9. Summary: DDP vs LQR

	LQR	DDP / iLQR
System	Linear ($Ax + Bu$)	Nonlinear ($f(x, u)$)
Solution	Global Optimum	Local Optimum
Process	One-shot	Iterative (Forward/Backward)
Result	Static Gain K	Trajectory $x(t), u(t) +$ Gains $K(t)$
Convergence	-	Quadratic (Newton-like)

Practical Note: DDP is the standard for Trajectory Optimization in modern robotics (e.g., walking robots, arm manipulation) because it is computationally efficient and handles nonlinearity well.